

รายงานโครงการ

CP413008 MACHINE LEARNING ENGINEERING FOR PRODUCTION

เรื่อง ระบบคัดกรองคำ ขอสินเชื่อ (Loan Approval MLOps)

สมาชิก	รหัสนักศึกษา	งานที่ส่งผ่าน PR	สไลด์ที่พูด
นายณภัทร โสตาตง	673380076-1	สคริปต์รัน test case ของอาจารย์	1-3
นายพุดพิงค์ พานิชพันธุ์	673380335-3	Rollback DAG + ความพร้อมวัน เสาร์	4-6 + คู่มือเครื่องตอนเสาร์
นายจักรภัทร เอี่ยมเสริม	673380309-4	รายงาน drift ด้วย Evidently	7-8
นายอภิมุข เพี้ยสักขา	673380354-9	เทสต์ monitoring + coverage ใน CI	9-10
นางสาวเตชินี ถามุลตรี	673380358-1	รายงานฉบับส่ง + สไลด์	11-13

สรุปผลงาน

ระบบคัดกรองคำ ขอสินเชื่อทำ งานได้ครบวงจร ตั้งแต่ข้อมูลดิบถึงการให้บริการ และส่งทั้งหมดได้ด้วยปุ่มเดียว ใน Airflow โมเดลได้ ROC-AUC 0.899
บนชุดทดสอบ ส่วน API มีค่า p95 เท่ากับ 67 ms ที่โหลด 20 คำขอ/ วินาที ผ่าน SLO 200 ms

- Repository: github.com/super-SuPer-dev/loan-mlops (private) ใช้ branch และ Pull Request และมี CI 5 job
- ชุดข้อมูล: Loan Approval Dataset 2026 (Kaggle, MIT) มี 30,000 คำขอ 36 คอลัมน์
- สิ่งที่น่าสนใจจริง: ข้อมูลเสียทำให้ pipeline หยุดพร้อมแจ้งเตือน, data drift และ concept drift ถูก ตรวจพบแยกกัน, retrain อัตโนมัติ, rollback ด้วยคำสั่งเดียว

1. การวางกรอบปัญหา

ระบบทำนายว่าคำขอสินเชื่อควรได้รับการอนุมัติหรือไม่ แล้วตัดสินใจอัตโนมัติเฉพาะคำขอที่มั่นใจ (ประมาณ 78% ของคำขอ) ส่วนที่เหลือส่งให้เจ้าหน้าที่สินเชื่อพิจารณา

ทำไมใช้ ML แทนการเขียนกฎ: การตัดสินใจขึ้นกับฟีเจอร์ 27 ตัวที่ส่งผลร่วมกัน เช่น คะแนนเครดิต ประวัติ ผิดนัด และอัตราส่วนหนี้ต่อรายได้ คะแนนเครดิตตัวเดียวแยกค่า ขอได้แค่ AUC 0.77 แต่โมเดลที่เรียนจากทุก ฟีเจอร์ได้ 0.89 นอกจากนี้เมื่อนโยบายธนาคารเปลี่ยน เทรนโมเดลใหม่ได้เองจากข้อมูลล่าสุด ไม่ต้องแก้กฎทีละข้อ

AI Project Canvas

องค์ประกอบ	รายละเอียด
Value proposition	ลดเวลาพิจารณาคำขอจากหลายวันเหลือไม่กี่วินาทีสำหรับคำขอที่ชัดเจน เจ้าหน้าที่เหลือเวลาดูแลเคสที่ก้ำกึ่ง
Stakeholders	ผู้ยื่นกู้เจ้าหน้าที่สินเชื่อฝ่ายบริหารความเสี่ยง ฝ่ายกำกับดูแล (ความเป็นธรรม)
Prediction task	Binary classification ของ Loan_Approved (Approved / Rejected) จากข้อมูลที่ลูกค้ากรอกตอนยื่นคำขอ
Decisions	อนุมัติอัตโนมัติ / ปฏิเสธอัตโนมัติ / ส่งเจ้าหน้าที่ (cascade ตามความมั่นใจ)
Data	ประวัติคำขอ 30,000 รายการ + ผลอนุมัติเฉลี่ยใหม่มาเมื่อพิจารณาเสร็จ
Features	ตัวเลข 17 + หมวดหมู่ 10 + อัตราส่วนที่สร้างเอง 2 (ตัด leakage และข้อมูลอ่อนไหวออก)
Building models	เทรนทุกสัปดาห์ + เมื่อเกิด concept drift เทรน 3 โมเดล ขนานกันใช้เวลาไม่ถึง 1 นาที
Making predictions	แบบ real-time ผ่าน /predict (ตอบทันทีที่หน้าเคาน์เตอร์/ แอป) + batch ผ่าน /predict/batch สำหรับคำขอค้าง
Impact simulation	ต้นทุนความผิดพลาด 3 : 1 ลดต้นทุนรวมจาก 1,879 เหลือ 1,446 (- 23%) เมื่อเทียบ threshold 0.5
Monitoring	Data drift (KS/Chi²), concept drift (AUC ตก), latency/error (Prometheus), สัดส่วนปฏิเสธแยกกลุ่ม

2. ตัวชี้วัด

Optimizing metric คือ ROC-AUC บน validation ส่วน gating metric มี 6 ข้อ โมเดลต้องผ่านทุกข้อจึง ขึ้นใช้งานได้ เกณฑ์ทุกตัวอยู่ใน src/config.py ที่เดียว

ประเภท	ตัวชี้วัด	เกณฑ์	เหตุผล
Optimizing	ROC-AUC (validation)	สูงสุด	วัดการเรียงความเสี่ยง ไม่ขึ้นกับ threshold คลาสค่อนข้างสมดุล (58/42)
Gating	ROC-AUC (test)	≥ 0.85	ต่ำกว่านี้คำขอที่ตัดสินใจอัตโนมัติจะน้อยจนไม่คุ้ม
Gating	Recall คลาสปฏิเสธ	≥ 0.80	กันอนุมัติผิดคน ซึ่งเสียหายมากกว่า

Gating	เทียบ champion เดิม	> -0.003	กันสลับไปโมเดลที่แย่ลง
Gating	p95 latency ต่อ 1 คำขอ	≤ 25 ms	มาจาก SLO 200 ms ที่ 20 req/s ให้ API ไม่ยุ่งเกิน 50%
Gating	ขนาดไฟล์โมเดล	≤ 50 MB	โหลดเร็วตอน reload / rollback
Gating	ส่วนต่างอัตราอนุมัติระหว่างเพศ	≤ 0.10	ความเป็นธรรม (ตรวจใน CI)

ตัวชี้วัดทางธุรกิจ เชื่อมกับโมเดลผ่าน threshold และโซนก้ำกึ่ง

- % คำขอที่ตัดสินใจอัตโนมัติ (ภาระงานเจ้าหน้าที่ที่ลดลง): 78.5% ที่ความแม่นยำ 80.3% ในส่วนนั้น
- ต้นทุนความผิดพลาดรวม (อนุมัติผิด 3 : ปฏิเสธผิด 1) ใช้เลือก threshold
- เวลาตอบกลับของลูกค้า (p95 ≤ 200 ms)

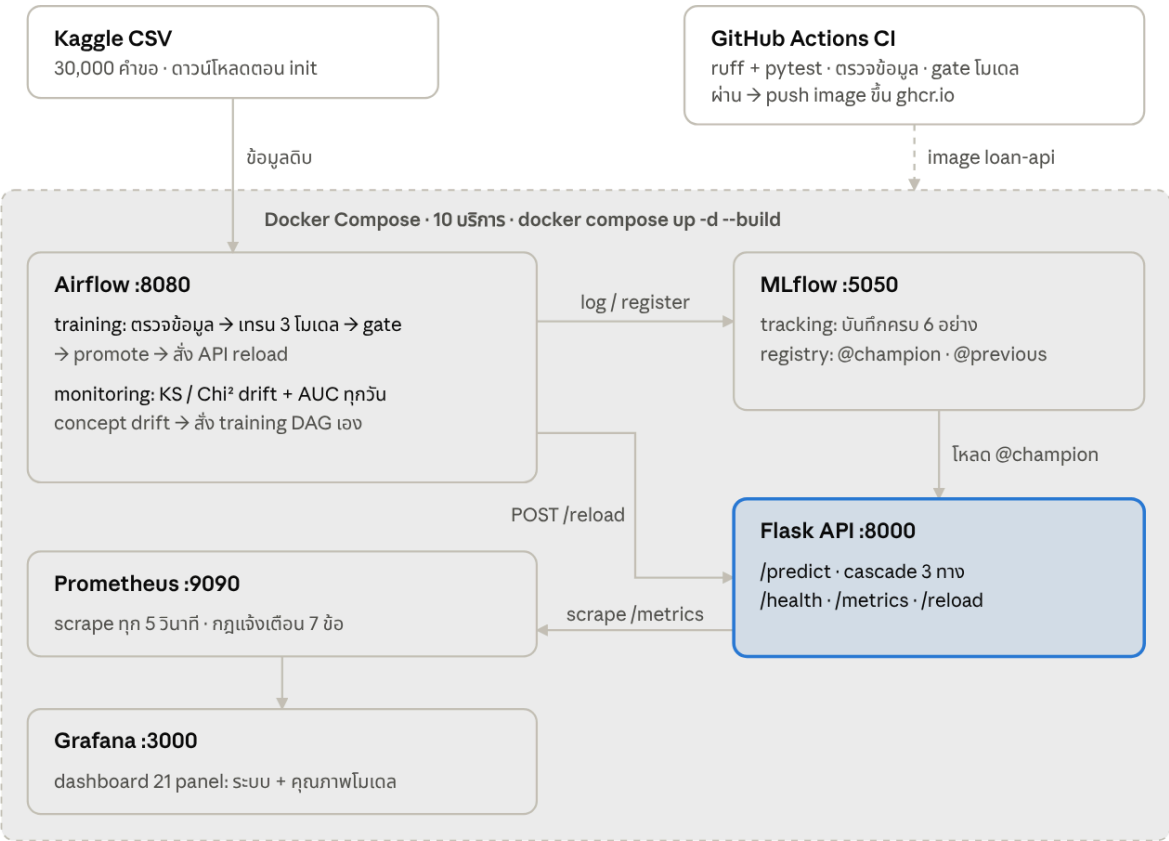
คำตอบคำถามตรวจสอบหัวข้อ

คำถาม	คำตอบ
ถ้าโมเดลทำนายผิด ใครเดือดร้อน	อนุมัติผิด=ธนาคารเสี่ยงหนี้เสีย-ปฏิเสธผิด=ลูกค้าเสียโอกาสและธนาคารเสียรายได้→ชดเชยด้วยต้นทุน 3:1และโซนส่งเจ้าหน้าที่
ตัวชี้วัดธุรกิจเชื่อมกับโมเดลอย่างไร	AUCสูง→คำขอที่มั่นใจได้มากขึ้น→%ตัดสินใจอัตโนมัติสูงขึ้น ส่วน recall ปฏิเสธคุมหนี้เสีย
ข้อมูลเปลี่ยนได้อย่างไร	Data drift:เศรษฐกิจถดถอยผู้สมัครรายได้/คะแนนต่ำมากขึ้น Concept drift:ธนาคารเปลี่ยนนโยบาย เช่นเลิกปล่อยกู้เกิน10ปี
ต้องตอบเร็วแค่ไหน รองรับเท่าใด	p95 ≤ 200 ms ที่ 20 คำ ขอ/วินาที (ช่วงพีคธนาคารขนาดกลาง) ความจุสูงสุดที่วัดได้ ≈ 75 คำ ขอ/วินาที
ถ้าเวอร์ชันใหม่แยกว่า รู้ได้อย่างไร ย้อนกลับอย่างไร	Evaluator เทียบกับ champion บน test ชุดเดียวกัน แยกว่า = ไม่ promote · หลุดหลัง deploy → rollback.py ย้าย alias กลับ @previous แล้ว API reload ทันที

3. สถาปัตยกรรมระบบ

ทุกบริการยกขึ้นด้วยคำสั่งเดียว และส่วนต่าง ๆ ส่งโมเดลต่อกันผ่าน MLflow registry เท่านั้น ไม่มีการคัดลอกไฟล์โมเดลด้วยมือ

ทุกส่วนรันใน Docker Compose และส่งต่อกันผ่าน MLflow registry



สถาปัตยกรรมระบบ - 8 ส่วนประกอบหลัก

Airflow เทรนและลงทะเบียนโมเดลใน MLflow แล้วสั่ง API โหลด @champion ใหม่ Prometheus ดึง metric จาก API รวมถึงผล monitoring DAG ล่าสุด ส่วน CI บน GitHub ตรวจทุก Pull Request ก่อน merge

เหตุผลการเลือกเครื่องมือ

หน้าที่	เครื่องมือ	เหตุผล
Version control	Git + GitHub	มี Pull Request และ Actions ในที่เดียว (Lab 9)
Containerization	Docker Compose	ยก 10 บริการด้วยคำ สั่งเดียวบนเครื่องเปล่า (Lab 8, 11)
Data validation	schema.json + src/validation.py	ข้อมูลเป็นตาราง ใช้ได้ทั้งใน DAG และใน API ไม่ต้องลง TensorFlow (Lab 11)

Experiment tracking + Registry	MLflow server	alias ทำ rollback ได้ด้วยการย้ายป้าย บันทึกครบ 6 อย่าง (Lab 9)
Orchestration	Airflow 3	branch, dynamic mapping, TriggerDagRun ปิดวงจร retrain (Lab 11)
Model serving	Flask	โมเดลเป็น sklearn ซึ่ง TF Serving ให้ บริการไม่ได้ ควบคุม cascade เอง (Lab 3, 7)
Monitoring	KS / Chi ² + Prometheus + Grafana	แยก drift สองแบบ + เฝ้าสถานะระบบ แบบ real-time (Lab 10, 11)
CI/CD	GitHub Actions	รันทุก PR ฟรี และ push image ขึ้น ghcr.io ได้เลย (Lab 9)

4. ข้อมูลและการตรวจสอบคุณภาพ

ข้อมูลสะอาด (ไม่มีค่าว่าง ไม่มีแถวซ้ำ อนุมิติ 57.6% / ปฏิเสธ 42.4%) แต่มี 4 คอลัมน์ที่รั่วคำตอบ ซึ่งเราตัด ออกก่อนเทรน

Data leakage

คอลัมน์	หลักฐาน	ตัดสินใจ
Approved_Amount	ผู้ถูกปฏิเสธมีค่า 0 ทุกแถว AUC คอลัมน์ เดียว = 1.000	ตัด (รู้หลังอนุมิติ)
Default_Risk	กลุ่ม High ถูกปฏิเสธ 95.1%	ตัด (ธนาคารประเมินทีหลัง)
Interest_Rate	ดอกเบี้ยที่กำ หนดให้หลังพิจารณา (AUC 0.70)	ตัด
Risk_Score	AUC คอลัมน์เดียว 0.925 มาจากระบบอื่นที่ไม่รู้สูตร	ตัด จากผล ablation
Gender , Marital_Status	ข้อมูลอ่อนไหว	ไม่ใช่เป็นฟีเจอร์ เก็บไว้ตรวจ fairness
Monthly_Income	เท่ากับ Annual_Income / 12 ทุกแถว	ใช้เป็นกฎตรวจสอบความ สอดคล้อง

การแบ่งข้อมูล

ข้อมูลไม่มีคอลัมน์เวลา จึงแบ่งแบบ stratified ด้วย seed 42 และบันทึก md5 ของไฟล์เป็นเวอร์ชันข้อมูล

(บังคับ line ending เป็น LF ให้ md5 เท่ากันทุกเครื่อง)

- 80% ข้อมูลอดีต (24,000) → train 70% / validation 15% / test 15%
- 20% pool (6,000) → จำลองคำ ขอ production 4 สัปดาห์ละ 1,500 และข้อมูลเสีย 2,000 แถว

Schema และการตรวจสอบผิดปกติ

Schema สร้างจาก train ครั้งเดียวแล้วเก็บเป็น schema.json (แนว Lab 11) มีกฎ 2 แบบ

- เรียนจากข้อมูล: ช่วง min-max เพื่อ 20%, หมวดที่เคยเห็น, คอลัมน์ที่บังคับ (เฉพาะที่ลูกค้ากรอกตอนยื่น)

- ความรู้โดเมน (ไม่เผื่อ): Credit_Score 300–850, อายุ ≥ 18 , เงินห้ามติดลบ, ค่าว่างต่อคอลัมน์ $\leq 5\%$, Application_ID ห้ามซ้ำ, Monthly_Income = Annual_Income / 12

สถิติข้อมูลเสีย: ไฟล์ bad_data.csv มี Credit_Score 920, รายได้ติดลบ, หมวด “Freelancer”, ค่าว่าง 15% และคอลัมน์ Collateral หาย ระบบจับได้ครบ 6 ปัญหา task example_validator ล้มด้วย AirflowFailException ทำให้ไม่มีการเทรนหรือ deploy และ on_failure_callback เขียน ALERT ลง alerts.jsonl (ส่ง webhook ได้ถ้าตั้งค่า) ฝั่ง API คำขอที่ผิด schema ได้ HTTP 400 พร้อมรายละเอียด

การแปลงข้อมูลและการกัน Training-Serving Skew

การแปลงทั้งหมดอยู่ใน sklearn Pipeline ก่อนเดียวกับโมเดล เขียนไว้ใน src/features.py ไฟล์เดียว ที่ทั้ง DAG และ API import

1. LoanFeatureEngineer : สร้าง Loan_To_Income , Liability_To_Asset แล้ว clip outlier ที่ quantile 1% / 99% ที่เรียนจาก train (ลบแถวทิ้งไม่ได้ เพราะต้องตอบทุกคำขอ)
2. ตัวเลข: SimpleImputer(median) $\rightarrow$ StandardScaler
3. หมวดหมู่: SimpleImputer(most_frequent) $\rightarrow$ OneHotEncoder(handle_unknown="ignore") หมวดใหม่ไม่ทำให้ error

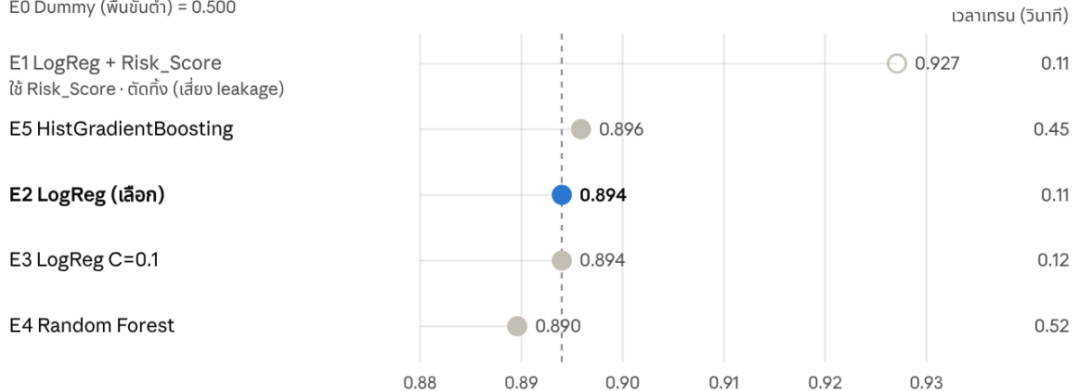
โมเดลถูกบันทึกพร้อมโค้ด src/ (MLflow code_paths) และ schema ที่ API ใช้ดาวน์โหลดจาก run เดียวกับที่เทรน เมื่อเทียบผลทำนายของโมเดลตอนเทรนกับโมเดลที่โหลดจาก registry ความต่างสูงสุดเท่ากับ 0 ทุกแถว และ AUC ผ่าน API บน 500 คำขอใหม่ = 0.900 เท่ากับตอนเทรน

5. การพัฒนาโมเดล

เลือก Logistic Regression (E2) เพราะแม่นยำใกล้เคียงโมเดลที่ซับซ้อนกว่า แต่เทรนและทำนายเร็วกว่า ไฟล์ เล็กกว่า และอธิบายเหตุผลผลการปฏิเสธได้ บน test set ได้ ROC-AUC 0.899

Logistic Regression แม่นใกล้เคียงโมเดลที่ซับซ้อนกว่า และเทรนเร็วกว่า 4 เท่า

ROC-AUC uu validation (4,500 คำขอ) · เส้นประ = โมเดลที่เลือก
E0 Dummy (พื้นขั้นต่ำ) = 0.500



แสดงการประเมินผลโมเดลต้นแบบบนชุด Validation (4,500 คำขอ) จาก **Exploratory Notebook**

(หมายเหตุ: ใน Production Pipeline จริงจะใช้ชุด Validation จำนวน 3,600 คำขอ ตามสัดส่วน Train 70% / Val 15% / Test 15%)

- E0 $\rightarrow$ E2: baseline หายคลาสที่เยอะสุดได้ AUC 0.500 โมเดลเรียนรู้ได้จริง
- ตัด Risk_Score (E1 เทียบ E2): เพิ่ม AUC 0.033 แต่เป็นคะแนนจากระบบอื่นที่ไม่รู้สูตร ถ้าสูตรเปลี่ยน โมเดลเราพึ่งตามโดยไม่รู้ตัว
- เลือกโมเดล: HistGradientBoosting ดีกว่าแค่ 0.002 Random Forest แยกกว่า และไฟล์ใหญ่ 50.6 MB ใน pipeline จริง select_best ตัดตัวที่เกินงบ latency ก่อน แล้วเลือกตาม AUC ถ้าต่างไม่ถึง 0.003 เลือกตัวที่ง่ายกว่า

Threshold ตามต้นทุน: อนุมัติผิดคนแพงกว่าปฏิเสธผิดคน 3 เท่า threshold ที่ต้นทุนต่ำสุดบน validation คือ 0.77 (notebook) / 0.72 (pipeline) ทำให้ recall ของคลาสปฏิเสธขึ้นจาก 0.73 เป็น 0.89–0.92

การอธิบายผล(Permutation Importance และ Coefficient)

ต้นทุนทางปฏิเสธ	coef	ต้นทุนทางอนุมัติ	coef
เคยผิดนัด (Defaulted)	-2.62	ประวัติชำระครบ (Paid Off)	+1.36
ว่างงาน (Unemployed)	-1.78	Credit_Score	+1.14
จ่ายช้าใน 2 ปี	-0.67	มีสินเชื่อยู่ (Active)	+0.57
Loan_To_Income	-0.60	ไม่เคยกู้ (No Previous Loan)	+0.57

Fairness: ไม่ได้ใช้เพศเป็นฟีเจอร์ อัตราอนุมัติที่ไม่แตกต่างกันระหว่างเพศ 0.017 (≤ 0.10) และ AUC ชาย 0.8967 / หญิง 0.8968

6. Experiment Tracking และ Model Registry

ทุกการเทรน (ทั้งใน notebook, DAG และ CI) บันทึกครบ 6 อย่างลง MLflow server และทุกโมเดลที่ผ่าน การประเมินถูกลงทะเบียนพร้อมสถานะ

สิ่งที่บันทึก	วิธี
เวอร์ชันโค้ด	tag git_commit อ่านจาก .git โดยตรง(ใน container ไม่มีคำสั่ง git) + airflow_run_id
เวอร์ชันข้อมูล / ไฮเปอร์พารามิเตอร์	param data_md5, data_source, n_train
ตัวชี้วัด	val AUC/F1/accuracy/recall ปฏิเสธ, p95 latency, test AUC, fairness gap
ไฟล์ผลลัพธ์	โมเดล (Pipeline เต็มก่อน + src/), schema.json, confusion matrix
สภาพแวดล้อม	environment.json (Python, OS, เวอร์ชันไลบรารี) + uv.lock

ทะเบียนโมเดล ใช้ MLflow alias แบบ Lab 9 และ tag บอกลักษณะ

- @champion ตัวที่ให้บริการ · @previous champion ก่อนหน้า (ไว้ rollback) · @challenger ตัวที่ไม่ผ่านด่าน
- tag status = approved / rejected / rolled_back พร้อม threshold , test_roc_auc , airflow_run_id

ด่านตรวจก่อนอนุมัติ: task blessing_gate เลือกเส้นทาง promote หรือ register_rejected ตาม ผล gating metric ทุกข้อ ตัวอย่างจริง: Random Forest ไม่ผ่านเพราะไฟล์ 50.6 MB เกินเกณฑ์ และ AUC แยกว่า champion 0.005

Rollback: scripts/rollback.py ย้าย @champion กลับไป @previous ติด tag rolled_back แล้วเรียก /reload สำเร็จแล้วจาก v3 กลับ v2 API โหลดเวอร์ชันเก่าทันทีโดยไม่ต้อง build หรือ restart container

Rollback ผ่านหน้าเว็บ Airflow: นอกจากสคริปต์ ยังกดย้อนได้จาก DAG loan_rollback_pipeline (rollback_champion → reload_api → notify) โดยไม่ต้องเปิด terminal เว้นช่อง to_version ว่าง = กลับไป @previous หรือระบุเวอร์ชันที่ต้องการได้ DAG นี้สั่งเองเท่านั้น (ไม่มีรอบอัตโนมัติ) ตั้ง max_active_runs = 1 กันกดซ้อนแล้ว alias สลับไปมา และตั้ง task rollback ไม่ retry เพราะการลองซ้ำจะสลับกลับไปเวอร์ชันเดิม task reload_api ตรวจสอบว่า API โหลดเวอร์ชันที่คาดหวังจริง ถ้าไม่ตรงให้ล้ม ส่วน notify ส่ง ALERT ระดับ WARNING ทุกครั้ง ต่างจากสคริปต์ตรงที่ Airflow เก็บบันทึกว่ากดเมื่อไรและผลเป็นอย่างไร

7. การให้บริการและ SLO

Flask API ใน Docker ผ่าน SLO: p95 = 66.7 ms ที่ 20 คำ ขอ/วินาที (เกณฑ์ 200 ms) และ

5xx = 0% เลือก Flask เพราะใช้ใน Lab 3 และ 7 และโมเดลเป็น sklearn ซึ่ง TF Serving ให้บริการไม่ได้

Endpoint	หน้าที่
GET/health	200 เมื่อโหนดโมเดลแล้ว, 503 เมื่อยังไม่มี (Docker healthcheck ทุก 15 วินาที)
POST/predict	ตรวจ schema → ทำนาย → ตัดสินแบบ cascade
POST/predict/batch	ทำนายหลายคำขอ (งานกลางคืน)
POST/reload	โหลด @champion ใหม่ (Airflow เรียกหลัง promote, rollback เรียกหลังย้าย alias)
GET/metrics	จำนวนคำขอ, latency histogram, ผลตัดสิน, เกณฑ์, ผล monitoring ล่าสุด

รูปแบบการให้บริการ: real-time + cascade (Lab 7) ลูก้ารอผลอยู่จึงต้องตอบทันที คำขอผิด schema ได้ 400 โดยไม่ทำนาย ส่วนความน่าจะเป็นที่ $\geq \text{threshold} + 0.10$ = อนุมัติอัตโนมัติ, $< \text{threshold} - 0.10$ = ปฏิเสธอัตโนมัติ, ที่เหลือส่งเจ้าหน้าที่ (human-in-the-loop)

ผลวัดจริง (scripts/load_test.py , คำขอจริงจาก pool ปนคำขอเสีย 5%)

โมเมนต์	p50	p95	Throughput	5xx
คงที่ 20 คำขอ/วินาที, 30 วินาที	23.7 ms	66.7 ms	20.0 คำขอ/วินาที	0%
Stress 10 ผู้ใช้พร้อมกัน (ความจุสูงสุด)	131.7 ms	186.3 ms	75.1 คำ ขอ/วินาที	0%
Stress 40 ผู้ใช้พร้อมกัน (สาธิต alert)	-	1,328 ms	35.0 คำ ขอ/วินาที	0%

Logs: ทุกคำขอ /predict เขียน JSON หนึ่งบรรทัด (เวลา, status, latency, request, response, เวอร์ชันโมเดล) ลง stdout และ logs/api/predictions.jsonl

สิ่งที่เจอระหว่างทดสอบ: ช่วงแรกด้าน latency ตั้งไว้ 50 ms หลวมเกินไป หลัง retrain ได้ HistGradientBoosting (~35 ms ต่อคำ ขอใน container) คำ ขอเริ่มต่อคิวจน p95 ขึ้นไป 195 ms เกือบเกิน SLO เราจึงผูกด้านกับ SLO (25 ms) และให้ select_best ตัดโมเดลที่เกินจน latency ก่อนเลือกตาม AUC หลังแก้ p95 หลัง retrain เหลือ 66.7 ms

8. Monitoring และนโยบาย Retrain

ระบบแยก data drift กับ concept drift ได้จริง ใน W2 พีเจอร์เปลี่ยนไป 63% แต่ AUC ยังดี จึงแค่แจ้งเตือน ส่วน W3 ข้อมูลเข้าไม่เปลี่ยนเลยแต่ AUC ตก 0.099 จึง retrain อัตโนมัติ

Data drift		Concept drift
อะไรเปลี่ยน	ข้อมูลเข้า $P(X)$	ความสัมพันธ์ $P(y X)$
ตรวจด้วย	KS test (ตัวเลข), Chi-square (หมวดหมู่) ไม่ต้องรอเฉลย	AUC บนเฉลยจริง เทียบ AUC ตอน deploy
เกณฑ์แจ้งเตือน	$p < 0.01$ ในเกิน 20% ของฟีเจอร์	AUC ตกเกิน 0.05
จำลองด้วย	เศรษฐกิจถดถอย: สุ่มแบบถ่วงน้ำหนักให้คน คะแนน/รายได้ต่ำมากขึ้น ($P(y X)$ เท่าเดิม)	ธนาคารเลิกปล่อยกู้ระยะเกิน 120 เดือน (เปลี่ยน label ไม่แตะข้อมูลเข้า)

ผลสาธิต (รันจากศูนย์ด้วย scripts/demo.py)

Batch	โมเดล	Drift share (KS/Chi ²)	AUC (ตอน deploy)	ผลลัพธ์
W1 ปกติ	v2 logreg	0%	0.895 (0.899)	ไม่มี alert
W2 data drift	v2 logreg	63%	0.918 (0.899)	ALERT data drift ไม่ retrain
W3 concept drift	v2 logreg	0%	0.801 (0.899)	ALERT concept drift → retrain → v3
W4 หลัง retrain	v3 hist_gboost	0%	0.931 (0.913)	ไม่มี alert

นโยบาย Retrain

เงื่อนไข	การกระทำ
AUC ตกเกิน 0.05 เมื่อเฉลยมาถึง	Monitoring DAG สั่ง training DAG ด้วย TriggerDagRunOperator เทรนด้วยข้อมูลล่าสุดที่มีเฉลย (sliding window) เพราะเฉลยเก่าไม่ตรงนโยบายแล้ว
Data drift เกิน 20% แต่ AUC ปกติ	แจ้งเตือนทีมให้ตรวจ ยังไม่ retrain
ข้อมูลเสีย	หยุด ห้าม retrain แจ้งเตือน
ตามรอบ	training DAG ทุกวันจันทร์ 02:00 UTC

โมเดลใหม่ทุกตัวต้องผ่าน gate เดิมและชนะ champion บน test ชุดเดียวกันก่อน promote

รายงาน Evidently: นอกจาก KS / Chi² ที่ให้ตัวเลขตัดสินใจอัตโนมัติ monitoring DAG มี task drift_report สร้างรายงาน data drift แบบ HTML ด้วย Evidently (DataDriftPreset) ให้คนดูว่า drift ที่ฟีเจอร์ไหนและการกระจายเปลี่ยนอย่างไร ใช้ 27 ฟีเจอร์ที่โมเดลใช้จริง และสุ่ม reference 5,000 แถวจากข้อมูลอดีตให้รายงานสร้างเร็ว task จะ log จำนวนและสัดส่วนคอลัมน์ที่ Evidently นับว่า drift เทียบกับสัดส่วนจาก KS / Chi² ของเรา รายงานเป็นส่วนเสริม ถ้าสร้างไม่

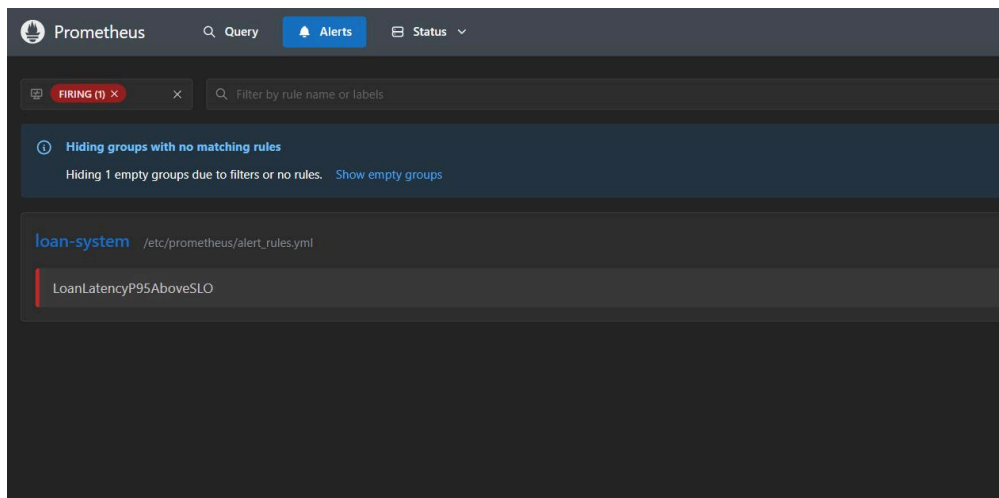
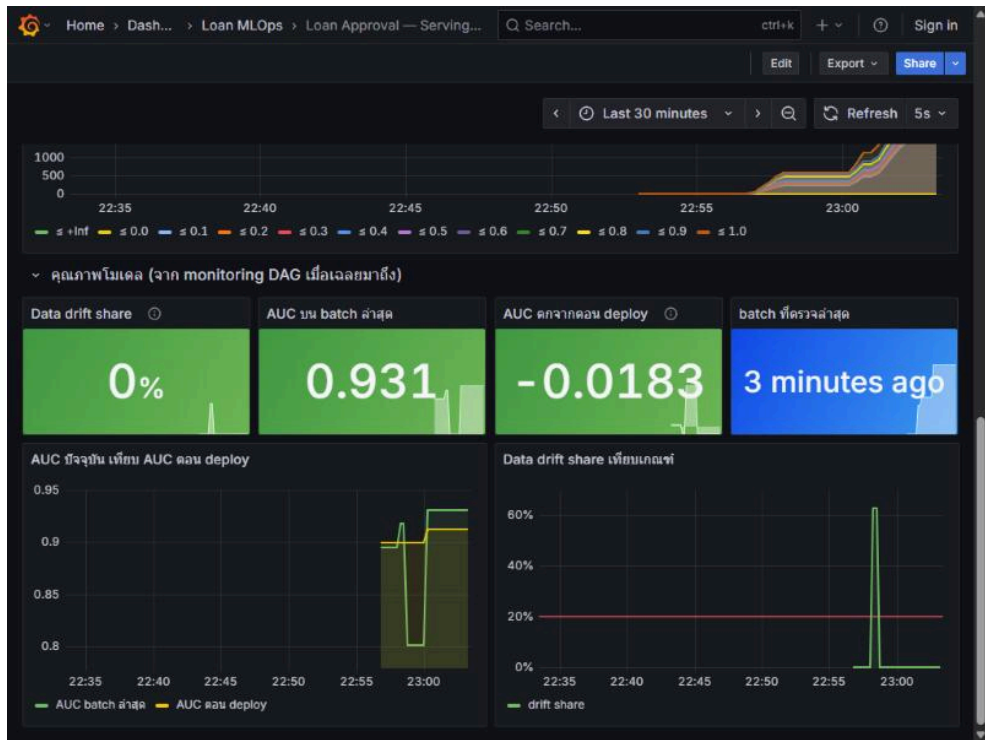
สำเร็จจะไม่ทำให้ DAG ล้ม และตัวเลขที่ใช้ตัดสินใจ alert / retrain ยังมาจาก check_batch มีเทสต์ 2 ข้อ คือข้อมูลที่มีการกระจายเดิม drift น้อยกว่า 20% และข้อมูล
ที่เลื่อนค่า (เศรษฐกิจถดถอยจำลอง) ถูกจับได้อย่างน้อย 3 คอลัมน์

Prometheus + Grafana

กฎแจ้งเตือน 7 ข้อ เกณฑ์อ่านจาก metric loan_config_threshold ที่ API ส่งออกจาก src/config.py

กฎ	ด้าน	เงื่อนไข
LoanApiDown	ระบบ	API ไม่ตอบ 30 วินาที
LoanModelNotLoaded	ระบบ	ยังไม่มีโมเดล 1 นาที
LoanLatencyP95AboveSLO	ระบบ	p95 > 200 ms (ทดสอบให้ firing แล้วที่ 1.9 วินาที)
LoanErrorRateAboveSLO	ระบบ	5xx > 1%
LoanDataDrift	โมเดล	drift share > 20%
LoanConceptDrift	โมเดล	AUC ตก > 0.05
LoanRejectRateSpike	โมเดล	AUTO_REJECT > 70% ใน 5 นาที (prediction drift ไม่ ต้องรอเฉลย)

Dashboard 21 panel(18 Data Panels + 3 Row Headers) แบ่งเป็นสถานะระบบ · การตัดสินใจของโมเดล · คุณภาพโมเดล metric เฉพาะเจาะจงคือ สัดส่วนปฏิเสธ
อัตโนมัติแยกตามสถานะการจ้างงาน ซึ่งเห็นว่ากลุ่ม Unemployed ถูกปฏิเสธ 94.9% และ Contract 62.1%



9. CI/CD, Pipeline และการทำ ซ้ำ

CI บน GitHub Actions ตรวจสอบครบ 3 ด้าน (คุณภาพโค้ด ข้อมูล โมเดล) และส่งมอบ image รอบแรกบน GitHub ผ่านครบ 5 job ภายใน 4.5 นาที

Job	ตรวจอะไร	รอบแรก
code-quality	ruff + pytest 14 เทสต์ (การแปลงข้อมูล, schema, API, registry, rollback)	ผ่าน 64 วินาที
data-validation	ดาวน์โหลดข้อมูลจริง ตรวจ schema และข้อมูลเสียต้องถูกจับได้ ≥ 5 ปัญหา	ผ่าน 24 วินาที

model-quality	เทรบบนข้อมูลจริง ต้องผ่านทุกข้อ + fairness	ผ่าน 33 วินาที
dag-tests	build image Airflow แล้วทดสอบ DAG 5 เทสต์ข้างใน	ผ่าน 97 วินาที
deliver	build image API, smoke test, push ขึ้น ghcr.io เมื่อ merge เข้า main	ผ่าน 98 วินาที

model-quality summary	
✖ Model quality gate (ข้อมูลจริง)	
รายการ	ผล
โมเดลที่เลือก	logreg
threshold	0.72
ROC-AUC (≥ 0.95)	0.8994
Recall ปฏิเสธ (≥ 0.8)	0.8885
p95 latency ms (≤ 25.0)	18.26
ขนาดโมเดล MB (≤ 50.0)	0.010
Fairness: ส่วนต่างสัดส่วนผิด (≤ 0.1)	0.0171
ผ่าน roc_auc	ไม่ผ่าน
ผ่าน recall_rejected	ผ่าน
ผ่าน p95_latency	ผ่าน
ผ่าน model_size	ผ่าน
ผ่าน vs_champion	ผ่าน
Job summary generated at run-time	
Annotations 1 error, 4 warnings, and 4 notices model-quality Process completed with exit code 1.	

Pipeline แบบ DAG (Airflow 3 ต่อยอด Lab 11) มี 3 DAG (2 DAG หลักซึ่งรันซ้ำทั้งกระบวนการได้ และ rollback DAG สั่งเอง)

- loan_training_pipeline (14 task): example_gen → schema_gen → example_validator → trainer × 3 (dynamic mapping ขนานกัน) → select_best → evaluator + fairness_audit → blessing_gate → promote / register_rejected → reload_api → report
- loan_monitoring_pipeline : check_batch → send_alerts → needs_retrain(short circuit) → trigger_training และอีกสาขา drift_report (รายงาน Evidently แบบ HTML ส่วนเสริม)
- loan_rollback_pipeline : rollback_champion → reload_api → notify (สั่งเองเท่านั้น ย้อน @champion แล้วสั่ง API โหลดใหม่)
- ตรวจทั้งหมดอยู่ใน src/ ไม่อิง Airflow จึงทดสอบด้วย pytest ได้ตรงๆ

ทำซ้ำได้ / รันจากเครื่องเปล่า: docker compose up -d --build ยก 10 บริการและดาวน์โหลดข้อมูลเอง โลบารี่ทุกตัวล็อกเวอร์ชันใน uv.lock และ requirements-ml.txt (เวอร์ชันเดียวกันทุก container) seed คงที่ 42 ทุกจุด เรียบทุกอย่างแล้วรันจากศูนย์ 2 ครั้ง ได้ md5 ข้อมูลและ test AUC (0.8994) เท่าเดิม

Git: ทำงานบน branch feature/* แล้วเปิด Pull Request เข้า main CI รันทุก PR

เทสต์และ coverage: ชุดเทสต์มี 38 ข้อ แบ่งเป็น 30 ข้อใน job code-quality (data 6, model 4, API 5, monitoring 6, data_prep 3, สคริปต์ test case 4, รายงาน Evidently 2) และ 8 ข้อใน tests/test_days.py ที่ต้องมี Airflow จึงรันใน job dag-tests และถูกข้ามอัตโนมัติบนเครื่องที่ไม่มี Airflow รันทั้งชุดแล้วผ่าน 30 ข้อ (ข้าม 1 ไฟล์) coverage ของ src/ รวม 85% (494 statements, ฟลัด 73) โดย config.py, features.py, monitoring.py ได้ 100%, drift_report.py และ validation.py 94%, steps.py 89%

สคริปต์ test case ของอาจารย์: scripts/run_test_cases.py รันไฟล์ CSV ผ่าน API จริงได้ทั้งกรณีปกติและกรณีผิดปกติ (วิธีใช้อยู่ใน docs/test_cases.md) ทำ 3 อย่าง คือ ตรวจทั้งไฟล์กับ schema แล้วบอกว่าคอลัมน์ไหนผิดอย่างไร, ส่งทีละแถวเข้า /predict (ได้ 200 พร้อมผลตัดสิน AUTO_APPROVE / AUTO_REJECT / MANUAL_REVIEW หรือ 400 พร้อมเหตุผล ซึ่งเป็นพฤติกรรมที่ถูกต้องของข้อมูลเสีย) และถ้าไฟล์มีคอลัมน์ Loan_Approved จะคำนวณ accuracy และ ROC-AUC ผลทั้งหมดเขียนลง reports/test_cases.json และมีเทสต์ 4 ข้อ

All pull requests

New pull request

is:pr state:closed

Open

1

Closed

7

AuthorLabelProjectsMilestonesReviewsAssigneeNewest

feat: รายงาน drift ด้วย Evidently

#14 · by jakrapatiam-create was merged 1 hour ago · 5 tasks done · 5/5

1

เพิ่มเทสต์ monitoring/data_prep และแสดง coverage ใน CI

#13 · by Tammy-AP was merged yesterday · 5 tasks done · 5/5

1

เพิ่มสคริปต์รัน test case ผ่าน API สำหรับวันนำเสนอ

#12 · by naphat67 was merged yesterday · 5/5

1

feat: ความพร้อมวันสาธิต (reset, .env.example, แจ้งเตือน Discord)

#11 · by super-SuPer-dev was merged 1 hour ago · 5 tasks done · 5/5

1

feat: Rollback DAG ย้อนเวอร์ชันโมเดลด้วยปุ่มเดียว

#10 · by super-SuPer-dev was merged 1 hour ago · 5 tasks done · 5/5

fix: cascade ไม่มี AUTO_APPROVE เมื่อ threshold สูง + เทสต์ latency ไม่ทิ้ง

#9 · by super-SuPer-dev was merged 3 days ago · 5/5

docs: หลักฐานภาพหน้าจอ + วิธีทำงานเป็นทีมผ่าน PR

#2 · by super-SuPer-dev was merged 3 days ago · 5/5

Commits

main

All usersAll time

Commits on Oct 3, 2026

เพิ่มรายงาน drift แบบ HTML ด้วย Evidently ใน monitoring DAG (#14)

jakrapatiam-create authored 1 hour ago · 5 / 5

Verifiede6f7c59

เพิ่มสคริปต์ reset ระบบ, .env.example และเช็กลิสต์วันสาธิต (#11)

super-SuPer-dev authored 1 hour ago · 5 / 5

Verified7fd0c96

เพิ่ม rollback DAG: ย้อน champion และสั่ง API reload จากหน้าเว็บ Airflow (#10)

super-SuPer-dev authored 1 hour ago · 5 / 5

Verified709c9b1

เพิ่มเทสต์ monitoring/data_prep และแสดง coverage ใน CI (#13)

Tammy-AP authored yesterday · 5 / 5

Verifiede2ae73c

เพิ่มสคริปต์รัน test case ผ่าน API สำหรับวันนำเสนอ (#12)

naphat67 authored yesterday · 5 / 5

Verifiede43e418

Commits on Oct 1, 2026

fix: keep AUTO_APPROVE reachable at high thresholds; stabilize latency in model tests (#9)

super-SuPer-dev authored 3 days ago · 5 / 5

Verifiedebf3645

docs: evidence screenshots, contribution guide and PR template (#2)

super-SuPer-dev authored 3 days ago · 5 / 5

Verifiedfb65c1b

Loan approval MLOps pipeline: Airflow, MLflow, Flask API, monitoring, CI

super-SuPer-dev committed 3 days ago · 5 / 5

a6e8264

10. การใช้ AI ช่วยงาน ข้อจำกัด และงานในอนาคต

กลุ่มใช้ Claude ช่วยเขียนโค้ดและร่างรายงานนี้ สมาชิกตรวจ รัน และอธิบายโค้ดทุกบรรทัดที่ส่งได้ ตามที่โจทย์กำหนด

ส่วนของงาน	AI ช่วยอะไร	สมาชิกตรวจ/ตัดสินใจอะไร
Notebook ต้นแบบ	ร่างโค้ดและคำอธิบายทุกชั้น	ตรวจสอบการติดต่อกลับ Data Leakage (Risk_Score, Approved_Amount) และเลือก

ส่วนของงาน	AI ช่วยอะไร	สมาชิกตรวจ/ตัดสินใจอะไร
		กำหนดอัตราส่วนต้นทุน 3 : 1
src/, DAG, Flask API	เขียนโค้ดโครงสร้าง MLOps ต่อ ยอดจาก Lab 9 และ 11	ปรับแต่ง Threshold cascade, ปรับ แต่ง Gating Metrics, และสร้าง Rollback DAG บน Airflow
Docker, Prometheus, Grafana, CI	ร่างไฟล์ Config และช่วยแก้ไข ปัญหา Port Conflict บน Windows	กำหนดกฎแจ้งเตือน 7 ข้อ ออกแบบ Grafana Dashboard 21 panels(18 + 3 row) และเพิ่ม Unit Test Coverage ใน CI จนถึง 85%
รายงานและเอกสาร	ร่างเนื้อหาจากผลที่รันจริง	ตรวจสอบความถูกต้องของตัวเลข สรุป ผลการทดสอบ และตรวจข้อความทั้ง ฉบับ

ข้อจำกัด

- ข้อมูลเป็นข้อมูลสังเคราะห์และไม่มีคอสมันเวลา drift ทั้งหมดจึงเป็นการจำลอง
- หลังเปลี่ยนนโยบาย โมเดลใหม่เทรนจาก 1,500 แถว และ test split มีแค่ 225 แถว ตัวเลขจึงกว้างได้มากกว่าโมเดลแรก
- Flask dev server ทำงานโปรเซสเดียว ความจุสูงสุด ≈ 75 คำขอ/วินาที
- การแจ้งเตือนเขียนลงไฟล์ + Prometheus ยังไม่มี Alertmanager ส่งออกช่องทางจริง (ตั้ง ALERT_WEBHOOK_URL ได้)

งานในอนาคต

- ประเมินผลโมเดลก่อนเผยแพร่มาถึงด้วย NannyML CBPE
- Canary release: ส่ง traffic 10% ไปที่ challenger ก่อน promote เต็ม
- ให้เหตุผลรายคำขอ (reason codes) จาก coefficient ส่งกลับไปกับผลปฏิเสธ